

R 과 외부 언어의 통합

R 프로그래밍 with workflow design



외부 언어와 통합하는 이유

- R의 성능 한계 보완
  - R은 인터프리터 언어로 반복문·대규모 계산 시 속도가 느림
  - **컴파일 언어(C/C++/Fortran)**와 연동해 연산 성능 개선
  - 예: Rcpp 사용 시 R보다 수십 배 빠른 함수 구현 가능
- 외부 생태계 활용
  - Python에는 머신러닝·딥러닝 패키지가 풍부 (예: `scikit-learn`, `PyTorch`)
  - `reticulate` 패키지를 통해 R에서 Python 코드 호출 가능
- 확장성과 협업
  - 외부 언어 기능을 통해 R의 기능 보완 및 확장 가능
  - 다양한 언어 사용자와 효율적인 협업 가능

Rcpp를 이용한 R과 C++의 연동

Rcpp란?

- Rcpp
  - R과 C++을 쉽게 연결해주는 인터페이스
  - R의 계산 속도 한계를 C++로 보완할 수 있음
  - 반복 루프, 행렬 계산 등 복잡한 연산을 빠르게 처리 가능
- 주요 장점
  - 계산 속도 향상: 수십·수백 배 빠름
  - 자료형 호환: R의 벡터와 행렬을 C++ 자료형 자동 변환
  - 기존 C++ 코드 및 라이브러리 활용 가능

Rcpp 사용을 위한 사전 준비

- Rcpp 패키지 설치: `install.packages("Rcpp")`
- 플랫폼별 컴파일러 설치
  - **Windows**: Rtools 설치
  - **Mac**: Xcode 설치
  - **Linux**: gcc 및 관련 패키지
- 참고 자료: Rcpp for Everyone 등

```
library(Rcpp)
```

C++ 함수 작성 구조

```
// [[Rcpp::export]]
RETURN_TYPE FUNCTION_NAME(ARGUMENT_TYPE ARGUMENT){
  // do something
  return RETURN_VALUE;
}
```

- `// [[Rcpp::export]]`: R에서 인식할 수 있도록 표시
- `sourceCpp("파일.cpp")` 로 실행 가능

R과 Rcpp의 자료형 매핑

R 자료형	Rcpp (scalar)	Rcpp (vector)	Rcpp (matrix)
`logical`	`bool`	`LogicalVector`	`LogicalMatrix`
`integer`	`int`	`IntegerVector`	`IntegerMatrix`
`numeric`	`double`	`NumericVector`	`NumericMatrix`
`character`	`string`	`CharacterVector`	`CharacterMatrix`

실행 방식 1: cppFunction()

```
cppFunction('
int add(int x, int y) {
  return x + y;
}')

# R에서 사용
add(2, 3)
```

```
[1] 5
```

- 간단한 코드용. 복잡한 코드 관리에는 부적절

실행 방식 2: sourceCpp()

- 두 실수에 곱을 구하는 multiply() 함수를 C++ 스크립트 my_code.cpp를 아래와 같이 작성

```
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
double multiply(double x, double y) {
  return x * y;
}
```

- R에서 사용

```
sourceCpp("multiply.cpp")
multiply(3, 4)
```

```
[1] 12
```

디버깅 팁: Rcout 사용

```cpp
// [[Rcpp::export]]
int add(int x, int y) {
  Rcout << "x = " << x << ", y = " << y << std::endl;
  return x + y;
}
```

- 실행 중 변수 값을 출력하여 디버깅
- 복잡한 디버깅은 `gdb`, `lldb` 같은 디버거 사용 가능

예제 개요: 유클리드 거리 행렬 계산

- **유클리드 거리(**Euclidean distance**)**는 두 점 사이의 직선 거리

$$d(p, q) = \sqrt{\sum_{k=1}^{d} (p_k - q_k)^2}, \quad p, q \in \mathbb{R}^d$$

- **유클리드 거리 행렬** D: 각 관측치 $p_1, ..., p_n \in \mathbb{R}^d$ 쌍의 거리를 저장하는 $n \times n$ 행렬

$$D_{ij} = d(p_i, p_j) = \sqrt{\sum_{k=1}^{d} (p_{i,k} - p_{j,k})^2}$$

- 응용 분야: 거리 기반 군집 분석, 차원 축소, 그래프 분석 등

R로 작성한 함수

```r
euclidean_distance_r = function(mat) {
  n = nrow(mat)
  dist_mat = matrix(0, n, n)
  for (i in 1:n) {
    for (j in 1:n) {
      dist_mat[i, j] = sqrt(sum((mat[i, ] - mat[j, ])^2))
    }
  }
  return(dist_mat)
}
```

- 직관적이지만 계산 시간이 오래 걸림 ($O(n^2 d)$)

C++로 작성한 Rcpp 함수

```
library(Rcpp)
cppFunction('
NumericMatrix euclidean_distance_cpp(NumericMatrix mat) {
  int n = mat.nrow();
  int d = mat.ncol();
  NumericMatrix dist_mat(n, n);
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      double sum_sq = 0.0;
      for (int k = 0; k < d; k++) {
        sum_sq += pow(mat(i, k) - mat(j, k), 2);
      }
      dist_mat(i, j) = sqrt(sum_sq);
    }
  }
  return dist_mat;
}
')
```

- 세미콜론, 변수 선언, 0-based 인덱스에 주의

테스트 데이터 생성

```r
set.seed(923)
mat = matrix(runif(300 * 10), nrow = 300, ncol = 10)

dist_mat1 = euclidean_distance_r(mat)
dist_mat2 = euclidean_distance_cpp(mat)
```

- 300개의 10차원 랜덤 벡터 생성
- 두 함수 결과 비교: `dist_mat1[1:5, 1:5]`, `dist_mat2[1:5, 1:5]`
- 결과 일치 확인됨

성능 비교 (microbenchmark)

```r
library(microbenchmark)
microbenchmark(
  R = euclidean_distance_r(mat),
  Rcpp = euclidean_distance_cpp(mat),
  times = 100
)
print(benchmark_results)
```

```
Unit: microseconds
expr        min         lq       mean     median         uq
   R  62040.708 62755.728 64451.9488 63618.6545 64132.671
Rcpp    760.099   768.709   804.7607   800.9965   819.672

               max neval
   R   102681.466   100
Rcpp     1253.165   100
```

- Rcpp로 약 **80배 빠른** 계산 가능

`reticulate`를 이용한 R과 Python의 연동

reticulate 개요

- `reticulate`는 R에서 Python 코드를 실행하고 객체를 주고받을 수 있게 해주는 패키지
- R과 Python의 데이터 분석, 머신러닝 도구를 통합 활용 가능
- 동일한 환경에서 두 언어의 장점을 모두 활용 가능

Python 환경 설정

- 기본 파이썬 해석기는 시스템 기본값 사용
- 특정 가상환경 또는 Conda 환경 지정 가능:

```
library(reticulate)

use_python("/usr/bin/python3")          # 시스템 파이썬
use_virtualenv("myenv")                 # virtualenv 사용
use_condaenv("my_conda_env")            # conda 환경 지정
```

Python 코드 실행

```
py_run_string("x = 5\ny = x**2")
py$x
```

```
## [1] 5
```

```
py$y
```

```
## [1] 25
```

- py_run_string() 으로 직접 문자열 실행 가능
- py$<name> 으로 Python 변수 접근 가능

Python 파일 실행

```r
py_run_file("myscript.py")
```

- Python 파일 전체를 실행할 수 있음
- R과 Python 간 객체 변환

Python 타입	R 타입
`int`	numeric
`float`	numeric
`bool`	logical
`str`	character
`list`	list
`dict`	named list
`numpy.array`	matrix / vector
`pandas.DataFrame`	data.frame

- 변환은 자동으로 처리되며, 필요 시 명시적 변환도 가능

예제: numpy와 연동

```
np = import("numpy")
a = np$array(c(1, 2, 3))
a + 10
```

```
## [1] 11 12 13
```

- `import()`를 통해 Python 패키지 직접 불러오기 가능
- numpy, pandas, sklearn 등 다양한 패키지 사용 가능

예제: scikit-learn 활용

```
skl_lr = import("sklearn.linear_model")

x = as.matrix(mtcars[,-1]) # 설명 변수와 반응 변수 설정
y = as.numeric(mtcars[, 1])

np = import("numpy") # 파이썬 객체 (numpy배열)로 변환
pyx = np$array(x)
pyy = np$array(y)

lr = skl_lr$LinearRegression() # 선형 회귀 모델 생성
lr$fit(pyx,pyy)

## LinearRegression()
coefficients = lr$coef_   # 계수와 절편 저장 및 출력
coefficients[1:4]

## [1] -0.11144048  0.01333524 -0.02148212  0.78711097
intercept = lr$intercept_
intercept

## [1] 12.30337
```